

1.MySQL的执行原理-2

1.1.MySQL的查询成本

1.1.1.什么是成本

MySQL执行一个查询可以有不同的执行方案，它会选择其中成本最低，或者说代价最低的那种方案去真正的执行查询。不过我们之前对成本的描述是非常模糊的，其实在MySQL中一条查询语句的执行成本是由下边这两个方面组成的：

I/O成本

我们的表经常使用的MyISAM、InnoDB存储引擎都是将数据和索引都存储到磁盘上的，当我们想查询表中的记录时，需要先把数据或者索引加载到内存中然后再操作。这个从磁盘到内存这个加载的过程损耗的时间称之为I/O成本。

CPU成本

读取以及检测记录是否满足对应的搜索条件、对结果集进行排序等这些操作损耗的时间称之为CPU成本。

对于InnoDB存储引擎来说，页是磁盘和内存之间交互的基本单位。

MySQL规定读取一个页面花费的成本默认是1.0（I/O成本）

读取以及检测一条记录是否符合搜索条件的成本默认是0.2（CPU成本）

1.0、0.2这些数字称之为成本常数，这两个成本常数我们最常用到，当然还有其他的成本常数。

注意，不管读取记录时需不需要检测是否满足搜索条件，哪怕是空数据，其成本都算是0.2。

1.1.2.单表查询的成本

1.1.2.1.基于成本的优化步骤实战

在一条单表查询语句真正执行之前，MySQL的查询优化器会找出执行该语句所有可能使用的方案，对比之后找出成本最低的方案，这个**成本最低的方案就是所谓的执行计划**，之后才会调用存储引擎提供的接口真正的执行查询，这个过程总结一下就是这样：

- 1、根据搜索条件，找出所有可能使用的索引
- 2、计算全表扫描的代价
- 3、计算使用不同索引执行查询的代价
- 4、对比各种执行方案的代价，找出成本最低的那一个

下边我们就以一个实例来分析一下这些步骤，单表查询语句如下：

```
SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S')
AND expire_time> '2021-03-22 18:28:28' AND expire_time<= '2021-03-22 18:35:09'
AND insert_time> expire_time AND order_note LIKE '%7*****排1%' AND order_status = 0;
```

看上去有点儿复杂，我们一步一步分析一下。

1. 根据搜索条件，找出所有可能使用的索引

我们前边说过，对于B+树索引来说，只要索引列和常数使用=、<=>、IN、NOT IN、IS NULL、IS NOT NULL、>、<、>=、<=、BETWEEN、!=（不等于也可以写成<>）或者LIKE操作符连接起来，就可以产生一个所谓的范围区间（LIKE匹配字符串前缀也行），MySQL把一个查询中可能使用到的索引称之为possible keys。

我们分析一下上边查询中涉及到的几个搜索条件：

order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S')，这个搜索条件可以使用二级索引idx_order_no。

expire_time> '2021-03-22 18:28:28' AND expire_time<= '2021-03-22 18:35:09'，这个搜索条件可以使用二级索引idx_expire_time。

insert_time> expire_time，这个搜索条件的索引列由于没有和常数比较，所以并不能使用到索引。

order_note LIKE '%hello%'，order_note即使有索引，但是通过LIKE操作符和以通配符开头的字符串做比较，不可以适用索引。

order_status = 0，由于该列上只有联合索引，而且不符合最左前缀原则，所以不会用到索引。

综上所述，上边的查询语句可能用到的索引，也就是possible keys只有idx_order_no,idx_expire_time。

```
EXPLAIN SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S')
AND expire_time> '2021-03-22 18:28:28' AND expire_time<= '2021-03-22 18:35:09'
AND insert_time> expire_time AND order_note LIKE '%7****排1%' AND order_status = 0;
```

id	select_type	table	partit type	possible_keys	key	key_len	ref	rows	filtered
1	SIMPLE	order_exp	(Null) range	idx_order_no idx_expire_time	idx_expire_time	5	(Null)	39	0.1

2. 计算全表扫描的代价

对于InnoDB存储引擎来说，全表扫描的意思就是把聚簇索引（主键索引）中的记录都依次和给定的搜索条件做一下比较，把符合搜索条件的记录加入到结果集，所以需要将聚簇索引对应的页面加载到内存中，然后再检测记录是否符合搜索条件。由于查询成本=I/O成本+CPU成本，所以计算全表扫描的代价需要两个信息：

1、聚簇索引占用的页面数

2、该表中的记录数

这两个信息从哪来呢？MySQL为每个表维护了一系列的统计信息，关于这些统计信息是如何收集起来的我们放在后边再说，现在看看怎么查看这些统计信息。

MySQL给我们提供了SHOW TABLE STATUS语句来查看表的统计信息，如果要看指定的某个表的统计信息，在该语句后加对应的LIKE语句就好了，比方说我们要查看order_exp这个表的统计信息可以这么写：

```
SHOW TABLE STATUS LIKE 'order_exp'\G
```

```

mysql> SHOW TABLE STATUS LIKE 'order_exp' \G
***** 1. row *****
      Name: order_exp
      Engine: InnoDB
      Version: 10
      Row_format: Dynamic
      Rows: 10350
      Avg_row_length: 153
      Data_length: 1589248
      Max_data_length: 0
      Index_length: 1212416
      Data_free: 4194304
      Auto_increment: 10819
      Create_time: 2022-06-17 13:20:19
      Update_time: NULL
      Check_time: NULL
      Collation: utf8_general_ci
      Checksum: NULL
      Create_options: row_format=DYNAMIC
      Comment:
1 row in set (0.01 sec)

```

出现了很多统计选项，但我们目前只需要两个：

Rows

本选项表示表中的记录条数。对于使用MyISAM存储引擎的表来说，该值是准确的，对于使用InnoDB存储引擎的表来说，该值是一个估计值。从查询结果我们也可以看出来，由于我们的order_exp表是使用InnoDB存储引擎的，所以虽然实际上表中有10567条记录，但是SHOW TABLE STATUS显示的Rows值只有10350条记录。但成本计算按照SHOW TABLE STATUS来计算。

Data_length

本选项表示表占用的存储空间字节数。使用MyISAM存储引擎的表来说，该值就是数据文件的大小，对于使用InnoDB存储引擎的表来说，该值就相当于聚簇索引占用的存储空间大小，也就是说可以这样计算该值的大小：

$\text{Data_length} = \text{聚簇索引的页面数量} \times \text{每个页面的大小}$

我们的order_exp使用默认16KB的页面大小，而上边查询结果显示Data_length的值是1589248，所以我们可以反向来推导出聚簇索引的页面数量：

聚簇索引的页面数量 = $1589248 \div 16 \div 1024 = 97$

我们现在已经得到了聚簇索引占用的页面数量以及该表记录数的估计值，所以就可以计算全表扫描成本了。

现在可以看一下全表扫描成本的计算过程：

I/O成本

$97 \times 1.0 + 1.1 = 98.1$

97指的是聚簇索引占用的页面数，1.0指的是加载一个页面的成本常数，后边的1.1是一个微调值。

关于这个微调值解释如下：

MySQL在真实计算成本时会进行一些微调，这些微调的值是直接硬编码到代码里的，没有注释而且这些微调的值十分的小，并不影响我们分析。

CPU成本

$10350 \times 0.2 + 1.0 = 2071$

10350指的是统计数据中表的记录数，对于InnoDB存储引擎来说是一个估计值，0.2指的是访问一条记录所需的成本常数，后边的1.0是一个微调值。

总成本：

$98.1 + 2071 = 2169.1$

综上所述，对于order_exp的全表扫描所需的总成本就是2169.1。

3. 计算使用不同索引执行查询的代价

从第1步分析我们得到，上述查询可能使用到idx_order_no，idx_expire_time这两个索引，我们需要分别分析单独使用这些索引执行查询的成本，最后还要分析是否可能使用到索引合并。

这里需要提一点的是，MySQL查询优化器先分析使用唯一二级索引的成本，再分析使用普通索引的成本，我们这里两个索引都是普通索引，先算哪个都可以。我们也先分析idx_expire_time的成本，然后再看使用idx_order_no的成本。

3.1使用idx_expire_time执行查询的成本分析

idx_expire_time对应的搜索条件是：

```
expire_time>'2021-03-22 18:28:28' AND expire_time<= '2021-03-22 18:35:09'
```

也就是说对应的范围区间就是：('2021-03-22 18:28:28', '2021-03-22 18:35:09')。

使用idx_expire_time搜索会使用用二级索引 + 回表方式的查询，MySQL计算这种查询的成本依赖两个方面的数据：

1、范围区间数量

不论某个范围区间的二级索引到底占用了多少页面，查询优化器认为读取索引的一个范围区间的I/O成本和读取一个页面是相同的。本例中使用idx_expire_time的范围区间只有一个，所以相当于访问这个范围区间的二级索引付出的I/O成本就是： $1 \times 1.0 = 1.0$

2、需要回表的记录数

优化器需要计算二级索引的某个范围区间到底包含多少条记录，对于本例来说就是要计算idx_expire_time在('2021-03-22 18:28:28', '2021-03-22 18:35:09')这个范围区间中包含多少二级索引记录，计算过程是这样的：

步骤1：先根据expire_time>'2021-03-22 18:28:28'这个条件访问一下idx_expire_time对应的B+树索引，找到满足expire_time>'2021-03-22 18:28:28'这个条件的第一条记录，我们把这条记录称之为区间最左记录。我们前头说过在B+数树中定位一条记录的过程是很快的，是常数级别的，所以这个过程的性能消耗是可以忽略不计的。

步骤2：然后再根据expire_time<='2021-03-22 18:35:09'这个条件继续从idx_expire_time对应的B+树索引中找出最后一条满足这个条件的记录，我们把这条记录称之为区间最右记录，这个过程的性能消耗也可以忽略不计的。

步骤3：如果区间最左记录和区间最右记录相隔不太远（在MySQL 5.7这个版本里，只要相隔不大于10个页面即可），那就可以精确统计出满足expire_time>'2021-03-22 18:28:28' AND expire_time<='2021-03-22 18:35:09'条件的二级索引记录条数。否则只沿着区间最左记录向右读10个页面，计算平均每个页面中包含多少记录，然后用这个平均值乘以区间最左记录和区间最右记录之间的页面数量就可以

了。

那么问题又来了，怎么估计区间最左记录和区间最右记录之间有多少个页面呢？解决这个问题还得回到B+树索引的结构中来。

我们假设区间**最左记录在页b中**，**区间最右记录在页c中**，那么我们计算区间最左记录和区间最右记录之间的页面数量就相当于计算页b和页c之间有多少页面，而它们父节点中记录的每一条目录项记录都对应一个数据页，所以计算页b和页c之间有多少页面就相当于计算它们父节点（也就是页a）中对应的目录项记录之间隔着几条记录。在一个页面中统计两条记录之间有几条记录的成本就很小了。

不过还有问题，如果页b和页c之间的页面实在太多，以至于页b和页c对应的目录项记录都不在一个父页面中怎么办？既然是树，那就继续递归，之前我们说过一个B+树有4层高已经很了不得了，所以这个统计过程也不是很耗费性能。

知道了如何统计二级索引某个范围区间的记录数之后，就需要回到现实问题中来，MySQL根据上述算法测得idx_expire_time在区间('2021-03-22 18:28:28', '2021-03-22 18:35:09')之间大约有39条记录。

```
explain SELECT * FROM order_exp WHERE expire_time> '2021-03-22 18:28:28' AND
expire_time<= '2021-03-22 18:35:09';
```

信息 结果1 概况 状态											
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	order_e	(Null)	range	idx_expire_time	idx_ex5		(Null)	39	100	Using ir

读取这39条二级索引记录需要付出的CPU成本就是：

$$39 \times 0.2 + 0.01 = 7.81$$

其中39是需要读取的二级索引记录条数，0.2是读取一条记录成本常数，0.01是微调。

在通过二级索引获取到记录之后，还需要干两件事儿：

1、根据这些记录里的主键值到聚簇索引中做回表操作

MySQL评估回表操作的I/O成本依旧很简单粗暴，他们认为每次回表操作都相当于访问一个页面，也就是说二级索引范围区间有多少记录，就需要进行多少次回表操作，也就是需要进行多少次页面I/O。我们上边统计了使用idx_expire_time二级索引执行查询时，预计有39条二级索引记录需要进行回表操作，所以回表操作带来的I/O成本就是：

$$39 \times 1.0 = 39$$

其中39是预计的二级索引记录数，1.0是一个页面的I/O成本常数。

2、回表操作后得到的完整用户记录，然后再检测其他搜索条件是否成立

回表操作的本质就是通过二级索引记录的主键值到聚簇索引中找到完整的用户记录，然后再检测除expire_time> '2021-03-22 18:28:28' AND expire_time<'2021-03-22 18:35:09'这个搜索条件以外的搜索条件是否成立。

因为我们通过范围区间获取到二级索引记录共39条，也就对应着聚簇索引中39条完整的用户记录，读取并检测这些完整的用户记录是否符合其余的搜索条件的CPU成本如下：

$$39 \times 0.2 = 7.8$$

其中39是待检测记录的条数，0.2是检测一条记录是否符合给定的搜索条件的成本常数。

所以本例中使用idx_expire_time执行查询的成本就如下所示：

I/O成本：

$1.0 + 39 \times 1.0 = 40.0$ (范围区间的数量 + 预估的二级索引记录条数)

CPU成本:

$39 \times 0.2 + 0.01 + 39 \times 0.2 = 15.61$ (读取二级索引记录的成本 + 读取并检测回表后聚簇索引记录的成本)

综上所述, 使用idx_expire_time执行查询的总成本就是:

$40.0 + 15.61 = 55.61$

3.2使用idx_order_no执行查询的成本分析

idx_order_no对应的搜索条件是:

```
order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S')
```

也就是说相当于3个单点区间。与使用idx_expire_time的情况类似, 我们也需要计算使用idx_order_no时需要访问的范围区间数量以及需要回表的记录数, 计算过程与上面类似, 我们不详列所有计算步骤和说明了。

范围区间数量

使用idx_order_no执行查询时很显然有3个单点区间, 所以访问这3个范围区间的二级索引付出的I/O成本就是:

$3 \times 1.0 = 3.0$

需要回表的记录数

由于使用idx_expire_time时有3个单点区间, 所以每个单点区间都需要查找一遍对应的二级索引记录数, 三个单点区间总共需要回表的记录数是58。

```
explain SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S');
```

信息 结果1 概况 状态											
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	order_e	(Null)	range	idx_order_no	idx_or152		(Null)	58	100	Using ir

读取这些二级索引记录的CPU成本就是: $58 \times 0.2 + 0.01 = 11.61$

得到总共需要回表的记录数之后, 就要考虑:

根据这些记录里的主键值到聚簇索引中做回表操作, 所需的I/O成本就是: $58 \times 1.0 = 58.0$

回表操作后得到的完整用户记录, 然后再比较其他搜索条件是否成立

此步骤对应的CPU成本就是:

$58 \times 0.2 = 11.6$

所以本例中使用idx_order_no执行查询的成本就如下所示:

I/O成本:

$3.0 + 58 \times 1.0 = 61.0$ (范围区间的数量 + 预估的二级索引记录条数)

CPU成本:

$58 \times 0.2 + 58 \times 0.2 + 0.01 = 23.21$ (读取二级索引记录的成本 + 读取并检测回表后聚簇索引记录的成本)

综上所述, 使用idx_order_no执行查询的总成本就是: $61.0 + 23.21 = 84.21$

3.3是否有可能使用索引合并 (Index Merge)

本例中有关order_no和expire_time的搜索条件是使用AND连接起来的, 而对于idx_order_no和idx_expire_time都是范围查询, 也就是说查找到的二级索引记录并不是按照主键值进行排序的, 并不满足使用Intersection索引合并的条件, 所以并不会使用索引合并。而且MySQL查询优化器计算索引合并成本的算法也比较麻烦。

4. 对比各种方案, 找出成本最低的那一个

下边把执行本例中的查询的各种可执行方案以及它们对应的成本列出来:

- 全表扫描的成本: 2148.7
- 使用idx_expire_time的成本: 55.61
- 使用idx_order_no的成本: 84.21

很显然, 使用idx_expire_time的成本最低, 所以当然选择idx_expire_time来执行查询。

最后请注意: MySQL的源码中对成本的计算实际要更复杂, 但是以上基本思想和算法是没问题的。

1.1.3.Explain与查询成本

1.1.3.1.EXPLAIN输出成本

前面我们已经对MySQL查询优化器如何计算成本有了比较深刻的了解。但是EXPLAIN语句输出中缺少了一个衡量执行计划好坏的重要属性—— **成本**。

不过MySQL已经为我们提供了一种查看某个执行计划花费的成本的方式:

在EXPLAIN单词和真正的查询语句中间加上FORMAT=JSON。

这样我们就可以得到一个json格式的执行计划, 里边包含该计划花费的成本, 比如这样:

```
explain format= json SELECT * FROM order_exp WHERE order_no IN ('DD00_6S',
'DD00_9S', 'DD00_10S') AND expire_time> '2021-03-22 18:28:28' AND expire_time<=
'2021-03-22 18:35:09' AND insert_time> expire_time AND order_note LIKE '%7排1%'
AND order_status = 0\G
```

```
mysql> explain format= json SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S') AND expire_time> '20
21-03-22 18:28:28' AND expire_time<= '2021-03-22 18:35:09' AND insert_time> expire_time AND order_note LIKE '%7排1%' AND orde
r_status = 0\G
***** 1. row *****
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "55.61"
    },
    "table": {
      "table_name": "order_exp",
      "access_type": "range",
      "possible_keys": [
        "idx_order_no",
        "idx_expire_time"
      ],
      "key": "idx_expire_time",
      "used_key_parts": [
        "expire_time"
      ],
      "key_length": "5",
      "rows_examined_per_scan": 39,
      "rows_produced_per_join": 0,
      "filtered": "0.13",
      "index_condition": "((('demo_explain`.`order_exp`.`expire_time` > '2021-03-22 18:28:28') and ('demo_explain`.`order_exp`
.expire_time` <= '2021-03-22 18:35:09')))",
      "cost_info": {
        "read_cost": "55.60",
        "eval_cost": "0.01",
        "prefix_cost": "55.61",
        "data_read_per_join": "24"
      }
    },
    "table": {
      "table_name": "order_exp",
      "access_type": "range",
      "possible_keys": [
        "idx_order_no",
        "idx_expire_time"
      ],
      "key": "idx_expire_time",
      "used_key_parts": [
        "expire_time"
      ],
      "key_length": "5",
      "rows_examined_per_scan": 39,
      "rows_produced_per_join": 0,
      "filtered": "0.13",
      "index_condition": "((('demo_explain`.`order_exp`.`expire_time` > '2021-03-22 18:28:28') and ('demo_explain`.`order_exp`
.expire_time` <= '2021-03-22 18:35:09')))",
      "cost_info": {
        "read_cost": "55.60",
        "eval_cost": "0.01",
        "prefix_cost": "55.61",
        "data_read_per_join": "24"
      }
    }
  },
  "table": {
    "table_name": "order_exp",
    "access_type": "range",
    "possible_keys": [
      "idx_order_no",
      "idx_expire_time"
    ],
    "key": "idx_expire_time",
    "used_key_parts": [
      "expire_time"
    ],
    "key_length": "5",
    "rows_examined_per_scan": 39,
    "rows_produced_per_join": 0,
    "filtered": "0.13",
    "index_condition": "((('demo_explain`.`order_exp`.`expire_time` > '2021-03-22 18:28:28') and ('demo_explain`.`order_exp`
.expire_time` <= '2021-03-22 18:35:09')))",
    "cost_info": {
      "read_cost": "55.60",
      "eval_cost": "0.01",
      "prefix_cost": "55.61",
      "data_read_per_join": "24"
    }
  }
}
```

这么多字段怎么解释，这里我用截图的方式解释一下

```
***** 1. row *****
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "55.61"
    },
    "table": {
      "table_name": "order_exp",
      "access_type": "range",
      "possible_keys": [
        "idx_order_no",
        "idx_expire_time"
      ],
      "key": "idx_expire_time",
      "used key parts": [

```

整个查询语句只有1个select 关键字，该关键字对应的id号为1

整个查询的执行成本预计为55.61

```
    "cost_info": {
      "read_cost": "55.60",
      "eval_cost": "0.01",
      "prefix_cost": "55.61",
      "data_read_per_join": "24"
    },
    "used_columns": [
      "id"
    ]
  }
}
```

成本明细

单独的查询表成本:read_cost+eval_cost

和连接查询有关的数据量，单位字节，这里没有用

1.1.3.2.Optimizer Trace

自认为比较牛逼的同学可能有这样的疑问：“我就觉得使用其他的执行方案比EXPLAIN输出的这种方案强，凭什么优化器做的决定和我想的不一样呢？为什么MySQL一定要全文扫描，不用索引呢？”，所以：在MySQL 5.6以及之后的版本中，MySQL提出了一个optimizertrace的功能，这个功能可以让我们方便的查看优化器生成执行计划的整个过程，这个功能的开启与关闭由系统变量optimizer_trace决定：

```
SHOW VARIABLES LIKE 'optimizer_trace';
```

```
mysql> SHOW VARIABLES LIKE 'optimizer_trace';
+-----+-----+
| Variable_name | Value                                |
+-----+-----+
| optimizer_trace | enabled=off,one_line=off          |
+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

可以看到enabled值为off，表明这个功能默认是关闭的。

如果想打开这个功能，必须首先把enabled的值改为on，就像这样：（注意这个开关是session级别）

```
SET optimizer_trace= 'enabled=on';
```

```
mysql> SET optimizer_trace= 'enabled=on';
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW VARIABLES LIKE 'optimizer_trace';
+-----+-----+
| Variable_name | Value                               |
+-----+-----+
| optimizer_trace | enabled=on,one_line=off           |
+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql>
```

one_line的值是控制输出格式的，如果为on那么所有输出都将在一行中展示，我们就保持其默认值为off。

当停止查看语句的优化过程时，把optimizertrace功能关闭

```
SET optimizer_trace="enabled=off";
```

注意：开启trace会影响mysql性能，所以只能临时分析sql** 使用，用完之后立即关闭。**

现在我们有一个搜索条件比较多的查询语句，它的执行计划如下：

```
explain
SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S')
AND expire_time> '2021-03-22 18:28:28' AND insert_time> '2021-03-22
18:35:09' AND order_note LIKE '%7****排1%';
```

可以看到该查询可能使用到的索引有3个u_idx_day_status,idx_order_no,idx_expire_time，那么为什么优化器最终选择了idx_order_no而不选择其他的索引或者直接全表扫描呢？这时候就可以通过optimizer trace功能来查看优化器的具体工作过程：（记得开启optimizer trace功能）

```
SELECT * FROM information_schema.OPTIMIZER_TRACE\G
```

然后我们就可以输入我们想要查看优化过程的查询语句，当该查询语句执行完成后，就可以到information_schema数据库下的OPTIMIZER_TRACE表中查看完整的优化过程。

```
mysql> explain SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S') AND expire_time> '2021-03-22 18:28:28' AND insert_time> '2021-03-22 18:35:09' AND order_note LIKE '%7****排1%';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp | NULL | range | u_idx_day_status,idx_order_no,idx_expire_time | idx_order_no | 152 | NULL | 58 | 2.78 | Using where |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> SELECT * FROM information_schema.OPTIMIZER_TRACE\G
+-----+
***** 1. row *****
QUERY: explain SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S') AND expire_time> '2021-03-22 18:28:28' AND insert_time> '2021-03-22 18:35:09' AND order_note LIKE '%7****排1%'
TRACE: {
  "steps": [
    {
      "join_preparation": {
        "select#": 1,
        "steps": [
          {
            "IN_uses_bisection": true
          },
          {
            "expanded_query": "/* select#1 */ select `order_exp`.`id` AS `id`,`order_exp`.`order_no` AS `order_no`,`order_exp`.`order_note` AS `order_note`,"
          }
        ]
      }
    }
  ]
}
```

```
mysql> SELECT * FROM information_schema.OPTIMIZER_TRACE\G
***** 1. row *****
QUERY: explain SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S') AND expire_time> '2021-03-22 18:28:28'
AND insert_time> '2021-03-22 18:35:09' AND order_note LIKE '%7排1%'
TRACE: {
  "steps": [
    {
      "join_preparation": {
```

分析查询语句是什么

```
mysql> SELECT * FROM information_schema.OPTIMIZER_TRACE\G
***** 1. row *****
QUERY: explain SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S') AND expire_time> '2021-03-22 18:28:28'
AND insert_time> '2021-03-22 18:35:09' AND order_note LIKE '%7排1%'
TRACE: {
  "steps": [
    {
      "join_preparation": {
        "select#": 1,
        "steps": [
          {
            "IN_uses_bisection": true
          },
          {
            "expanded_query": "/* select#1 */ select `order_exp`.`id` AS `id`,`order_exp`.`order_no` AS `order_no`,`order_exp`.`order_note` AS `order_note`,`order_exp`.`insert_time` AS `insert_time`,`order_exp`.`expire_duration` AS `expire_duration`,`order_exp`.`expire_time` AS `expire_time`,`order_exp`.`order_status` AS `order_status` from `order_exp` where ((`order_exp`.`order_no` in ('DD00_6S','DD00_9S','DD00_10S')) and (`order_exp`.`expire_time` > '2021-03-22 18:28:28') and (`order_exp`.`insert_time` > '2021-03-22 18:35:09') and (`order_exp`.`order_note` like '%7排1%'))"
          }
        ]
      },
      "join_optimization": {
        "select#": 1,
        "steps": [
```

类似于查询优化器将查询语句重写后的语句

```
      "join_optimization": {
        "select#": 1,
        "steps": [
          {
            "condition_processing": {
              "condition": "WHERE",
              "original_condition": "((`order_exp`.`order_no` in ('DD00_6S','DD00_9S','DD00_10S')) and (`order_exp`.`expire_time` > '2021-03-22 18:28:28') and (`order_exp`.`insert_time` > '2021-03-22 18:35:09') and (`order_exp`.`order_note` like '%7排1%'))",
              "steps": [
                {
                  "transformation": "equality_propagation",
                  "resulting_condition": "((`order_exp`.`order_no` in ('DD00_6S','DD00_9S','DD00_10S')) and (`order_exp`.`expire_time` > '2021-03-22 18:28:28') and (`order_exp`.`insert_time` > '2021-03-22 18:35:09') and (`order_exp`.`order_note` like '%7排1%'))"
                },
                {
                  "transformation": "constant_propagation",
                  "resulting_condition": "((`order_exp`.`order_no` in ('DD00_6S','DD00_9S','DD00_10S')) and (`order_exp`.`expire_time` > '2021-03-22 18:28:28') and (`order_exp`.`insert_time` > '2021-03-22 18:35:09') and (`order_exp`.`order_note` like '%7排1%'))"
                },
                {
                  "transformation": "trivial_condition_removal",
                  "resulting_condition": "((`order_exp`.`order_no` in ('DD00_6S','DD00_9S','DD00_10S')) and (`order_exp`.`expire_time` > '2021-03-22 18:28:28') and (`order_exp`.`insert_time` > '2021-03-22 18:35:09') and (`order_exp`.`order_note` like '%7排1%'))"
                }
              ]
            },
            "substitute_generated_columns": {
            },
            "table_dependencies": [
              {
                "table": "`order_exp`",
                "row may be null": false
              }
            ]
          }
        ]
      }
    ]
  }
}
```

```
    },
    {
      "group_index_range": {
        "chosen": false,
        "cause": "not_group_by_or_distinct"
      },
      "analyzing_range_alternatives": {
        "range_scan_alternatives": [
          {
            "index": "u_idx_day_status",
            "ranges": [
              "0x99a92d28c9 < insert_time"
            ],
            "index_dives_for_eq_ranges": true,
            "rowid_ordered": false,
            "using_mrr": false,
            "index_only": false,
            "rows": 5175,
            "cost": 6211,
            "chosen": false,
            "cause": "cost"
          },
          {
            "index": "idx_order_no",
            "ranges": [
              "DD00_10S <= order_no <= DD00_10S",
              "DD00_6S <= order_no <= DD00_6S",
              "DD00_9S <= order_no <= DD00_9S"
            ],
            "index_dives_for_eq_ranges": true,
            "rowid_ordered": false,
            "using_mrr": false,
            "index_only": false,
            "rows": 58,
            "cost": 72.61,
            "chosen": true
          }
        ]
      }
    ]
  }
}
```

分析各种可能使用索引的成本

```
    }  
  ]  
}  
{  
{  
},  
{  
  "join_explain": {  
    "select#": 1,  
    "steps": [  
      ]  
    }  
  }  
]  
]  
  
MISSING_BYTES_BEYOND_MAX_MEM_SIZE: 0  
INSUFFICIENT_PRIVILEGES: 0  
row in set (0.00 sec)
```

执行阶段

优化过程大致分为了三个阶段:

prepare阶段

optimize阶段

execute阶段

我们所说的基于成本的优化主要集中在optimize阶段，对于单表查询来说，我们主要关注optimize阶段的"rows_estimation"这个过程，这个过程深入分析了对单表查询的各种执行方案的成本；

```

{
  "rows_estimation": [
    {
      "table": "`order_exp`",
      "range_analysis": {
        "table_scan": {
          "rows": 10350,
          "cost": 2169.1
        },
        "potential_range_indexes": [
          {
            "index": "PRIMARY",
            "usable": false,
            "cause": "not_applicable"
          },
          {
            "index": "u_idx_day_status",
            "usable": true,
            "key_parts": [
              "insert_time",
              "order_status",
              "expire_time"
            ]
          }
        ]
      },
      {
        "index": "idx_order_no",
        "usable": true,
        "key_parts": [
          "order_no",
          "id"
        ]
      }
    ]
  },
  {
    "index": "idx_order_no",
    "usable": true,
    "key_parts": [
      "order_no",
      "id"
    ]
  }
]
}

```

对于多表连接查询来说，我们更多需要关注"considered_execution_plans"这个过程，这个过程里会写明各种不同的连接方式所对应的成本。反正优化器最终会选择成本最低的那种方案来作为最终的执行计划，也就是我们使用EXPLAIN语句所展现出的那种方案。

```

{
  }
]
},
{
  "considered_execution_plans": [
    {
      "plan_prefix": [
      ],
      "table": "`order_exp`",
      "best_access_path": {
        "considered_access_paths": [
          {
            "rows_to_scan": 58,
            "access_type": "range",
            "range_details": {
              "used_index": "idx_order_no"
            },
            "resulting_rows": 1.611,
            "cost": 84.21,
            "chosen": true
          }
        ]
      },
      "condition_filtering_pct": 100,
      "rows_for_plan": 1.6109,
      "cost_for_plan": 84.21,
      "chosen": true
    }
  ]
},
{
  "attaching_conditions_to_tables": {

```

×

查找内容(N):

方向

☒ 向上(U)
 ☐ 向下(D)

查找下一个(F)

取消

☐ 区分大小写(C)

如果对使用EXPLAIN语句展示出的对某个查询的执行计划很不理解，就可以尝试使用optimizer trace功能来详细了解每一种执行方案对应的成本。

1.1.4.连接查询的成本

1.1.4.1.Condition filtering介绍

我们前边说过，MySQL中连接查询采用的是嵌套循环连接算法，驱动表会被访问一次，被驱动表可能会被访问多次，所以对于两表连接查询来说，它的查询成本由下边两个部分构成：

1、单次查询驱动表的成本

2、多次查询被驱动表的成本（具体查询多少次取决于对驱动表查询的结果集中有多少条记录）

对驱动表进行查询后得到的记录条数称之为驱动表的 扇出（英文名：fanout）。

很显然驱动表的扇出值越小，对被驱动表的查询次数也就越少，连接查询的总成本也就越低。当查询优化器想计算整个连接查询所使用的成本时，就需要计算出驱动表的扇出值，有的时候扇出值的计算是很容易的，比如下边这两个查询：

查询一：

```
SELECT * FROM order_exp AS s1 INNER JOIN order_exp2 AS s2;
```

假设使用s1表作为驱动表，很显然对驱动表的单表查询只能使用全表扫描的方式执行，驱动表的扇出值也很明确，那就是驱动表中有多少记录，扇出值就是多少。统计数据中s1表的记录行数是10573，也就是说优化器就直接会把10573当作在s1表的扇出值。

查询二：

```
SELECT * FROM order_exp AS s1 INNER JOIN order_exp2 AS s2 WHERE s1.expire_time>
'2021-03-22 18:28:28' AND s1.expire_time<= '2021-03-22 18:35:09';
```

仍然假设s1表是驱动表的话，很显然对驱动表的单表查询可以使用idx_expire_time索引执行查询。此时范围区间('2021-03-22 18:28:28','2021-03-22 18:35:09')中有多少条记录，那么扇出值就是多少。

但是有的时候扇出值的计算就变得很棘手，比方说下边几个查询：

查询三：

```
SELECT * FROM order_exp AS s1 INNER JOIN order_exp2 AS s2 WHERE s1.order_note >
'xyz';
```

本查询和查询一类似，只不过对于驱动表s1多了一个order_note > 'xyz'的搜索条件。查询优化器又不会真正的去执行查询，所以它只能猜这10573记录里有多少条记录满足order_note > 'xyz'条件。

查询四：

```
SELECT * FROM order_exp AS s1 INNER JOIN order_exp2 AS s2 WHERE
s1.expire_time>'2021-03-22 18:28:28' AND s1.expire_time<= '2021-03-22 18:35:09'
AND s1.order_note > 'xyz';
```

本查询和查询二类似，只不过对于驱动表s1也多了一个order_note > 'xyz'的搜索条件。不过因为本查询可以使用idx_expire_time索引，所以只需要从符合二级索引范围区间的记录中猜有多少条记录符合order_note > 'xyz'条件，也就是只需要猜在39条记录中有多少符合order_note > 'xyz'条件。

```
mysql>
mysql> explain SELECT * FROM order_exp AS s1 INNER JOIN order_exp2 AS s2 WHERE s1.expire_time> '2021-03-22 18:28:28' AND s1.expire_time<= '2021-03-22 18:35:09' ;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	s1	NULL	range	idx_expire_time	idx_expire_time	5	NULL	39	100.00	Using index condition
1	SIMPLE	s2	NULL	ALL	NULL	NULL	NULL	NULL	10324	100.00	Using join buffer (Block Nested Loop)

通过上面4个案例大家可以看到，MySQL很多时候在计算数据的时候很多时候只能靠猜。

MySQL把这个猜的过程称之为 **condition filtering**。当然，这个过程可能会使用到索引，也可能使用到统计数据，也可能就是MySQL单纯的瞎猜，整个评估过程非常复杂，所以我们不去细讲。

在MySQL 5.7之前的版本中，查询优化器在计算驱动表扇出时，如果是使用全表扫描的话，就直接使用表中记录的数量作为扇出值，如果使用索引的话，就直接使用满足范围条件的索引记录条数作为扇出值。

在MySQL 5.7中，MySQL引入了这个condition filtering的功能，就是还要猜一猜剩余的那些搜索条件能把驱动表中的记录再过滤多少条，其实本质上就是为了让成本估算更精确。我们所说的纯粹瞎猜其实是很不严谨的，MySQL称之为启发式规则。

1.1.4.2.两表连接的成本分析

连接查询的成本计算公式是这样的：

连接查询总成本 = 单次访问驱动表的成本 + 驱动表扇出数 × 单次访问被驱动表的成本

对于左（外）连接和右（外）连接查询来说，它们的驱动表是固定的，所以想要得到最优的查询方案只需要分别为驱动表和被驱动表选择成本最低的访问方法。

可是对于内连接来说，驱动表和被驱动表的位置是可以互换的，所以需要考虑两个方面的问题：

不同的表作为驱动表最终的查询成本可能是不同的，也就是需要考虑最优的表连接顺序。然后分别为驱动表和被驱动表选择成本最低的访问方法。

很显然，计算内连接查询成本的方式更麻烦一些，下边我们就以内连接为例来看看如何计算出最优的连接查询方案。当然在某些情况下，左（外）连接和右（外）连接查询在某些特殊情况下可以被优化为内连接查询。

我们来看看内连接，比如对于下边这个查询来说：

```
SELECT
    *
FROM
    order_exp AS s1
    INNER JOIN order_exp2 AS s2 ON s1.order_no = s2.order_note
WHERE s1.expire_time > '2021-03-22 18:28:28'
AND s1.expire_time <= '2021-03-22 18:35:09'
AND s2.expire_time > '2021-03-22 18:35:09'
AND s2.expire_time <= '2021-03-22 18:35:59';
```

可以选择的连接顺序有两种：

s1连接s2，也就是s1作为驱动表，s2作为被驱动表。

s2连接s1，也就是s2作为驱动表，s1作为被驱动表。

查询优化器需要分别考虑这两种情况下的最优查询成本，然后选取那个成本更低的连接顺序以及该连接顺序下各个表的最优访问方法作为最终的查询计划。我们定性的分析一下，不像分析单表查询那样定量的分析了：

具体都可以使用分析语句来执行

```
explain format=json    SQL语句
```

1.1.4.4.多表连接的成本分析

首先要考虑一下多表连接时可能产生出多少种连接顺序：

对于两表连接，比如表A和表B连接只有 AB、BA这两种连接顺序。其实相当于 $2 \times 1 = 2$ 种连接顺序。

对于三表连接，比如表A、表B、表C进行连接有ABC、ACB、BAC、BCA、CAB、CBA这6种连接顺序。其实相当于 $3 \times 2 \times 1 = 6$ 种连接顺序。

对于四表连接的话，则会有 $4 \times 3 \times 2 \times 1 = 24$ 种连接顺序。对于n表连接的话，则有 $n \times (n-1) \times (n-2) \times \dots \times 1$ 种连接顺序，就是n的阶乘种连接顺序，也就是n!。

有n个表进行连接，MySQL查询优化器要每一种连接顺序的成本都计算一遍么？那可是n!种连接顺序呀。其实真的是要都算一遍，不过MySQL用了很多办法减少计算非常多种连接顺序的成本的方法：

提前结束某种顺序的成本评估

MySQL在计算各种链接顺序的成本之前，会维护一个全局的变量，这个变量表示当前最小的连接查询成本。如果在分析某个连接顺序的成本时，该成本已经超过当前最小的连接查询成本，那就压根儿不对该连接顺序继续往下分析了。比方说A、B、C三个表进行连接，已经得到连接顺序ABC是当前的最小连接成本，比方说10.0，在计算连接顺序BCA时，发现B和C的连接成本就已经大于10.0时，就不再继续往后分析BCA这个连接顺序的成本了。

系统变量optimizer_search_depth

为了防止无穷无尽的分析各种连接顺序的成本，MySQL提出了optimizer_search_depth系统变量，如果连接表的个数小于该值，那么就继续穷举分析每一种连接顺序的成本，否则只对与optimizer_search_depth值相同数量的表进行穷举分析。很显然，该值越大，成本分析的越精确，越容易得到好的执行计划，但是消耗的时间也就越长，否则得到不是很好的执行计划，但可以省掉很多分析连接成本的时间。

```
mysql> show variables like 'optimizer_search_depth';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| optimizer_search_depth | 62 |
+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

根据某些规则压根儿就不考虑某些连接顺序

即使是有上边两条规则的限制，但是分析多个表不同连接顺序成本花费的时间还是会很长，所以MySQL干脆提出了一些所谓的启发式规则（就是根据以往经验指定的一些规则），凡是不满足这些规则的连接顺序压根儿就不分析，这样可以极大的减少需要分析的连接顺序的数量，但是也可能造成错失最优的执行计划。他们提供了一个系统变量optimizer_prune_level来控制到底是不是用这些启发式规则。

```
mysql> show variables like 'optimizer_prune_level';
```

Variable_name	Value
optimizer_prune_level	1

```
1 row in set, 1 warning (0.00 sec)
```

1.1.5.调节成本常数

我们前边已经介绍了两个成本常数：

读取一个页面花费的成本默认是1.0

检测一条记录是否符合搜索条件的成本默认是0.2

其实除了这两个成本常数，MySQL还支持很多，它们被存储到了MySQL数据库的两个表中：

```
SHOW TABLES FROM mysql LIKE '%cost%';
```

```
mysql> SHOW TABLES FROM mysql LIKE '%cost%';
```

Tables_in_mysql (%cost%)
engine_cost
server_cost

```
2 rows in set (0.00 sec)
```

因为一条语句的执行其实是分为两层的：server层、存储引擎层。

在server层进行连接管理、查询缓存、语法解析、查询优化等操作，在存储引擎层执行具体的数据存取操作。也就是说一条语句在server层中执行的成本是和它操作的表使用的存储引擎是没关系的，所以关于这些操作对应的成本常数就存储在了server_cost表中，而依赖于存储引擎的一些操作对应的成本常数就存储在了engine_cost表中。

1.1.5.1.mysql.server_cost表

server_cost表中在server层进行的一些操作对应的成本常数，具体内容如下：

```
SELECT * FROM mysql.server_cost;
```

```
mysql> SELECT * FROM mysql.server_cost;
```

cost_name	cost_value	last_update	comment
disk_temptable_create_cost	NULL	2021-10-11 15:00:34	NULL
disk_temptable_row_cost	NULL	2021-10-11 15:00:34	NULL
key_compare_cost	NULL	2021-10-11 15:00:34	NULL
memory_temptable_create_cost	NULL	2021-10-11 15:00:34	NULL
memory_temptable_row_cost	NULL	2021-10-11 15:00:34	NULL
row_evaluate_cost	NULL	2021-10-11 15:00:34	NULL

6 rows in set (0.00 sec)

我们先看一下server_cost各个列都分别是什么意思：

cost_name

表示成本常数的名称。

cost_value

表示成本常数对应的值。如果该列的值为NULL的话，意味着对应的成本常数会采用默认值。

last_update

表示最后更新记录的时间。

comment

注释。

从server_cost中的内容可以看出来，目前在server层的一些操作对应的成本常数有以下几种：

disk_temptable_create_cost 默认值40.0

创建基于磁盘的临时表的成本，如果增大这个值的话会让优化器尽量少的创建基于磁盘的临时表。

disk_temptable_row_cost 默认值1.0

向基于磁盘的临时表写入或读取一条记录的成本，如果增大这个值的话会让优化器尽量少的创建基于磁盘的临时表。

key_compare_cost 0.1

两条记录做比较操作的成本，多用在排序操作上，如果增大这个值的话会提升filesort的成本，让优化器可能更倾向于使用索引完成排序而不是filesort。

memory_temptable_create_cost 默认值2.0

创建基于内存的临时表的成本，如果增大这个值的话会让优化器尽量少的创建基于内存的临时表。

memory_temptable_row_cost 默认值0.2

向基于内存的临时表写入或读取一条记录的成本，如果增大这个值的话会让优化器尽量少的创建基于内存的临时表。

row_evaluate_cost 默认值0.2

这个就是我们之前一直使用的检测一条记录是否符合搜索条件的成本，增大这个值可能让优化器更倾向于使用索引而不是直接全表扫描。

MySQL在执行诸如DISTINCT查询、分组查询、Union查询以及某些特殊条件下的排序查询都可能在内部先创建一个临时表，使用这个临时表来辅助完成查询（比如对于DISTINCT查询可以建一个带有UNIQUE索引的临时表，直接把需要去重的记录插入到这个临时表中，插入完成之后的记录就是结果集了）。在数据量大的情况下可能创建基于磁盘的临时表，也就是为该临时表使用MyISAM、InnoDB等存储引擎，在数据量不大时可能创建基于内存的临时表，也就是使用Memory存储引擎。大家可以看到，创建临时表和对这个临时表进行写入和读取的操作代价还是很高的就行了。

这些成本常数在server_cost中的初始值都是NULL，意味着优化器会使用它们的默认值来计算某个操作的成本，如果我们想修改某个成本常数的值的话，需要做两个步骤：

对我们感兴趣的成本常数做update更新操作，然后使用下边语句即可：

```
FLUSH OPTIMIZER_COSTS;
```

当然，在你修改完某个成本常数后想把它们再改回默认值的话，可以直接把cost_value的值设置为NULL，再使用FLUSH OPTIMIZER_COSTS语句让系统重新加载。

1.1.5.2.mysql.engine_cost表

engine_cost表中在存储引擎层进行的一些操作对应的成本常数，具体内容如下：

```
SELECT * FROM mysql.engine_cost;
```

```
mysql> SELECT * FROM mysql.engine_cost;
```

engine_name	device_type	cost_name	cost_value	last_update	comment
default	0	io_block_read_cost	NULL	2021-10-11 15:00:34	NULL
default	0	memory_block_read_cost	NULL	2021-10-11 15:00:34	NULL

2 rows in set (0.00 sec)

与server_cost相比，engine_cost多了两个列：

engine_name列

指成本常数适用的存储引擎名称。如果该值为default，意味着对应的成本常数适用于所有的存储引擎。

device_type列

指存储引擎使用的设备类型，这主要是为了区分常规的机械硬盘和固态硬盘，不过在MySQL 5.7.X这个版本中并没有对机械硬盘的成本和固态硬盘的成本作区分，所以该值默认是0。

我们从engine_cost表中的内容可以看出来，目前支持的存储引擎成本常数只有两个：

io_block_read_cost 默认值1.0

从磁盘上读取一个块对应的成本。请注意我使用的是块，而不是页这个词。对于InnoDB存储引擎来说，一个页就是一个块，不过对于MyISAM存储引擎来说，默认是以4096字节作为一个块的。增大这个值会加重I/O成本，可能让优化器更倾向于选择使用索引执行查询而不是执行全表扫描。

memory_block_read_cost 默认值1.0

与上一个参数类似，只不过衡量的是从内存中读取一个块对应的成本。

怎么从内存中和从磁盘上读取一个块的默认成本是一样的？这主要是因为MySQL目前的实现中，并不能准确预测某个查询需要访问的块中有哪些块已经加载到内存中，有哪些块还停留在磁盘上，所以MySQL简单的认为不管这个块有没有加载到内存中，使用的成本都是1.0。

与更新server_cost表中的记录一样，我们也可以通过更新engine_cost表中的记录来更改关于存储引擎的成本常数，做法一样。

1.2.MySQL的查询重写规则

对于一些执行起来十分耗费性能的语句，MySQL还是依据一些规则，竭尽全力的把这个很糟糕的语句转换成某种可以比较高效执行的形式，这个过程也可以被称作查询重写。

1.2.1.条件化简

我们编写的查询语句的搜索条件本质上是一个表达式，这些表达式可能比较繁杂，或者不能高效的执行，MySQL的查询优化器会为我们简化这些表达式。

1.2.1.1.移除不必要的括号

有时候表达式里有许多无用的括号，比如这样：

```
((a = 5 AND b =c) OR ((a > c) AND (c < 5)))
```

看着就很烦，优化器会把那些用不到的括号给干掉，就是这样：

```
(a = 5 and b =c) OR (a > c AND c < 5)
```

1.2.1.2.常量传递 (constant_propagation)

有时候某个表达式是某个列和某个常量做等值匹配，比如这样：

```
a = 5
```

当这个表达式和其他涉及列a的表达式使用AND连接起来时，可以将其他表达式中的a的值替换为5，比如这样：

```
a = 5 AND b >a
```

就可以被转换为：

```
a = 5 AND b >5
```

等值传递 (equality_propagation)

有时候多个列之间存在等值匹配的关系，比如这样：

```
a = b and b = c and c = 5
```

这个表达式可以被简化为：

```
a = 5 and b = 5 and c = 5
```

1.2.1.3.移除没用的条件 (trivial_condition_removal)

对于一些明显永远为TRUE或者FALSE的表达式，优化器会移除掉它们，比如这个表达式：

```
(a < 1 and b= b) OR (a = 6 OR 5 != 5)
```

很明显， $b = b$ 这个表达式永远为TRUE， $5 \neq 5$ 这个表达式永远为FALSE，所以简化后的表达式就是这样的：

```
(a < 1 and TRUE) OR (a = 6 OR FALSE)
```

可以继续被简化为

```
a < 1 OR a =6
```

1.2.1.4.表达式计算

在查询开始执行之前，如果表达式中只包含常量的话，它的值会被先计算出来，比如这个：

```
a = 5 + 1
```

因为 $5 + 1$ 这个表达式只包含常量，所以就会被化简成：

```
a = 6
```

但是这里需要注意的是，如果某个列并不是以单独的形式作为表达式的操作数时，比如出现在函数中，出现在某个更复杂表达式中，就像这样：

```
ABS(a) > 5
```

或者：

```
-a < -8
```

优化器是不会尝试对这些表达式进行化简的。我们前边说过只有搜索条件中索引列和常数使用某些运算符连接起来才可能使用到索引，所以如果可以的话，最好让索引列以单独的形式出现在表达式中。

1.2.1.5.常量表检测

MySQL觉得下边这种查询运行的特别快：

使用主键等值匹配或者唯一二级索引列等值匹配作为搜索条件来查询某个表。

MySQL觉得这两种查询花费的时间特别少，少到可以忽略，所以也把通过这两种方式查询的表称之为常量表（英文名：constant tables）。优化器在分析一个查询语句时，先首先执行常量表查询，然后把查询中涉及到该表的条件全部替换成常数，最后再分析其余表的查询成本，比方说这个查询语句：

```

SELECT
    *
FROM
    table1
INNER JOIN table2 ON table1.column1 = table2.column2
WHERE
    table1.primary_key = 1;

```

很明显，这个查询可以使用主键和常量值的等值匹配来查询table1表，也就是在这个查询中table1表相当于常量表，在分析对table2表的查询成本之前，就会执行对table1表的查询，并把查询中涉及table1表的条件都替换掉，也就是上边的语句会被转换成这样：

```

SELECT
    table1记录的各个字段的常量值，
    table2.*
FROM
    table1
INNER JOIN table2 ON table1表column1列的常量值 = table2.column2;

```

1.2.2.外连接消除

我们前边说过，内连接的驱动表和被驱动表的位置可以相互转换，而左（外）连接和右（外）连接的驱动表和被驱动表是固定的。这就导致内连接可能通过优化表的连接顺序来降低整体的查询成本，而外连接却无法优化表的连接顺序。

我们之前说过，外连接和内连接的本质区别就是：对于外连接的驱动表的记录来说，如果无法在被驱动表中找到匹配ON子句中的过滤条件的记录，那么该记录仍然会被加入到结果集中，对应的被驱动表记录的各个字段使用NULL值填充；而内连接的驱动表的记录如果无法在被驱动表中找到匹配ON子句中的过滤条件的记录，那么该记录会被舍弃。查询效果就是这样：

```
SELECT * FROM e1 INNER JOIN e2 ON e1.m1 = e2.m2;
```

```

mysql> SELECT * FROM e1 INNER JOIN e2 ON e1.m1 = e2.m2;
+-----+-----+-----+-----+
| m1    | n1    | m2    | n2    |
+-----+-----+-----+-----+
|      2 | b     |      2 | b     |
|      3 | c     |      3 | c     |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

```
SELECT * FROM e1 LEFT JOIN e2 ON e1.m1 = e2.m2;
```

```
mysql> SELECT * FROM e1 INNER JOIN e2 ON e1.m1 = e2.m2;
+-----+-----+-----+-----+
| m1    | n1    | m2    | n2    |
+-----+-----+-----+-----+
|      2 | b     |      2 | b     |
|      3 | c     |      3 | c     |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

对于上边例子中的（左）外连接来说，由于驱动表e1中m1=1, n1='a'的记录无法在被驱动表e2中找到符合ON子句条件e1.m1 = e2.m2的记录，所以就直接把这条记录加入到结果集，对应的e2表的m2和n2列的值都设置为NULL。

因为凡是不符合WHERE子句中条件的记录都不会参与连接。只要我们在搜索条件中指定关于被驱动表相关列的值不为NULL，那么外连接中在被驱动表中找不到符合ON子句条件的驱动表记录也就被排除出最后的结果集了，也就是说：在这种情况下：外连接和内连接也就没有什么区别了！

另外再说下这个查询：

```
SELECT * FROM e1 LEFT JOIN e2 ON e1.m1 = e2.m2 WHERE e2.n2 IS NOT NULL
```

信息	结果1	概况	状态
m1	n1	m2	n2
▶	2 b	2 b	
	3 c	3 c	

由于指定了被驱动表e2的n2列不允许为NULL，所以上边的e1和e2表的左（外）连接查询和内连接查询是一样的。当然，我们也可以不用显式的指定被驱动表的某个列IS NOT NULL，只要隐含的有这个意思就行了，比方说这样：

```
SELECT * FROM e1 LEFT JOIN e2 ON e1.m1 = e2.m2 WHERE e2.m2 = 2
```

信息	结果1	概况	状态
m1	n1	m2	n2
2 b		2 b	

在这个例子中，我们在WHERE子句中指定了被驱动表e2的m2列等于2，也就相当于间接的指定了m2列不为NULL值，所以上边的这个左（外）连接查询其实和下边这个内连接查询是等价的：

```
SELECT * FROM e1 INNER JOIN e2 ON e1.m1 = e2.m2 WHERE e2.m2 = 2
```

我们把这种在外连接查询中，指定的WHERE子句中包含被驱动表中的列不为NULL值的条件称之为空值拒绝（英文名：reject-NULL）。在被驱动表的WHERE子句符合空值拒绝的条件后，外连接和内连接可以相互转换。这种转换带来的好处就是查询优化器可以通过评估表的不同连接顺序的成本，选出成本最低的那种连接顺序来执行查询。

1.2.3.子查询优化

1.2.3.1.子查询语法

在一个查询语句A里的某个位置也可以有另一个查询语句B，这个出现在A语句的某个位置中的查询B就被称为子查询，A也被称之为外层查询。子查询可以在一个外层查询的各种位置出现，比如：

1) SELECT子句中

也就是我们平时说的查询列表中，比如这样：

```
SELECT(SELECT m1 FROM e1 LIMIT 1);
```

其中的(SELECT m1 FROM e1LIMIT 1)就是子查询。

2) FROM子句中

```
SELECT m, n FROM (SELECT m2 + 1 AS m, n2 AS n FROM e2 WHERE m2 > 2) AS t;
```

这个例子中的子查询是：(SELECT m2+1 AS m, n2 AS n FROM e2 WHERE m2 > 2)

这里可以把子查询的查询结果当作是一个表，子查询后边的AS t表明这个子查询的结果就相当于一个名称为t的表，这个名叫t的表的列就是子查询结果中的列，比如例子中表t就有两个列：m列和n列。

这个放在FROM子句中的子查询本质上相当于一个表，但又和我们平常使用的表有点儿不一样，MySQL把这种由子查询结果集组成的表称之为 **派生表**。

3) WHERE**或ON子句中**

把子查询放在外层查询的WHERE子句或者ON子句中可能是我们最常用的一种使用子查询的方式了，比如这样：

```
SELECT * FROM e1 WHERE m1 IN (SELECT m2 FROM e2)
```

这个查询表明我们想要将(SELECT m2FROM e2)这个子查询的结果作为外层查询的IN语句参数，整个查询语句的意思就是我们想找e1表中的某些记录，这些记录的m1列的值能在e2表的m2列找到匹配的值。

4) ORDER BY**子句、GROUP BY子句中**

虽然语法支持，但没啥意义。

还有一些其他的子查询，这里不一一列举

1.2.3.2.子查询在MySQL中是怎么执行的

想象中子查询的执行方式是这样的：

如果该子查询是不相关子查询，比如下边这个查询：

```
SELECT * FROM s1 WHERE order_note IN (SELECT order_note FROM s2);
```

先单独执行(SELECT order_note FROM s2)这个子查询。

然后在将上一步子查询得到的结果当作外层查询的参数再执行外层查询

最后根据子查询的查询结果来检测外层查询WHERE子句的条件是否成立，如果成立，就把外层查询的那条记录加入到结果集，否则就丢弃。

但真的是这样吗？其实MySQL用了一系列的办法来优化子查询的执行，大部分情况下这些优化措施其实挺有效的，下边我们来看看各种不同类型的子查询具体是怎么执行的。

不同的子查询

1) 按返回的结果集区分子查询

因为子查询本身也算是一个查询，所以可以按照它们返回的不同结果集类型而把这些子查询分为不同的类型：

1.1) 标量子查询

那些只返回一个单一值的子查询称之为标量子查询，比如这样：

```
SELECT (SELECT m1 FROM e1 LIMIT 1);
```

或者这样：

```
SELECT * FROM e1 WHERE m1 = (SELECT MIN(m2) FROM e2);
```

```
SELECT * FROM e1 WHERE m1 < (SELECT MIN(m2) FROM e2);
```

这两个查询语句中的子查询都返回一个单一的值，也就是一个标量。这些标量子查询可以作为一个单一值或者表达式的一部分出现在查询语句的各个地方。

1.2) 行子查询

顾名思义，就是返回一条记录的子查询，不过这条记录需要包含多个列（只包含一个列就成了标量子查询了）。比如这样：

```
SELECT * FROM e1 WHERE (m1, n1) = (SELECT m2, n2 FROM e2 LIMIT 1);
```

其中的(SELECT m2, n2 FROM e2 LIMIT 1)就是一个行子查询，整条语句的含义就是要从e1表中找一些记录，这些记录的m1和n1列分别等于子查询结果中的m2和n2列。

1.3) 列子查询

列子查询自然就是查询出一个列的数据，不过这个列的数据需要包含多条记录（只包含一条记录就成了标量子查询了）。比如这样：

```
SELECT * FROM e1 WHERE m1 IN (SELECT m2 FROM e2);
```

其中的(SELECT m2 FROM e2)就是一个列子查询，表明查询出e2表的m2列的值作为外层查询IN语句的参数。

1.4) 表子查询

顾名思义，就是子查询的结果既包含很多条记录，又包含很多个列，比如这样：

```
SELECT * FROM e1 WHERE (m1, n1) IN (SELECT m2, n2 FROM e2);
```

其中的(SELECT m2, n2 FROM e2)就是一个表子查询，

这里需要和行子查询对比一下，行子查询中我们用了LIMIT 1来保证子查询的结果只有一条记录，表子查询中不需要这个限制。

2)按与外层查询关系来区分子查询

2.1)不相关子查询

如果子查询可以单独运行出结果，而不依赖于外层查询的值，我们就可以把这个子查询称之为不相关子查询。我们前边介绍的那些子查询全部都可以看作不相关子查询。

2.2)相关子查询

如果子查询的执行需要依赖于外层查询的值，我们就可以把这个子查询称之为相关子查询。比如：

```
SELECT * FROM e1 WHERE m1 IN (SELECT m2 FROM e2 WHERE n1 = n2);
```

例子中的子查询是(SELECT m2 FROM e2 WHERE n1 = n2),

可是这个查询中有一个搜索条件是 $n1 = n2$ ，别忘了 $n1$ 是表 $e1$ 的列，也就是外层查询的列，也就是说子查询的执行需要依赖于外层查询的值，所以这个子查询就是一个相关子查询。

3) [NOT] IN/ANY/SOME/ALL子查询

对于列子查询和表子查询来说，它们的结果集中包含很多条记录，这些记录相当于是一个集合，所以就不能单纯的和另外一个操作数使用操作符来组成布尔表达式了，MySQL通过下面的语法来支持某个操作数和一个集合组成一个布尔表达式：

3.1)IN或者NOT IN

具体的语法形式如下：

操作数 [NOT] IN (子查询)

这个布尔表达式的意思是用来判断某个操作数在不在由子查询结果集组成的集合中，比如下边的查询的意思是找出 $e1$ 表中的某些记录，这些记录存在于子查询的结果集中：

```
SELECT * FROM e1 WHERE (m1, n1) IN (SELECT m2, n2 FROM e2);
```

3.2) ANY/SOME (ANY和SOME是同义词)

具体的语法形式如下：

操作数 比较符 ANY/SOME(子查询)

这个布尔表达式的意思是只要子查询结果集中存在某个值和给定的操作数做比较操作，比较结果为TRUE，那么整个表达式的结果就为TRUE，否则整个表达式的结果就为FALSE。比方说下边这个查询：

```
SELECT * FROM e1 WHERE m1 > ANY(SELECT m2 FROM e2);
```

这个查询的意思就是对于 $e1$ 表的某条记录的 $m1$ 列的值来说

如果子查询(SELECT $m2$ FROM $e2$)的结果集中存在一个小于 $m1$ 列的值，那么整个布尔表达式的值就是TRUE，

否则为FALSE，也就是说只要 $m1$ 列的值大于子查询结果集中最小的值，整个表达式的结果就是TRUE，所以上边的查询本质上等价于这个查询：

```
SELECT * FROM e1 WHERE m1 > (SELECT MIN(m2) FROM e2);
```

另外， $=ANY$ 相当于判断子查询结果集中是否存在某个值和给定的操作数相等，它的含义和IN是相同的。

3.3)ALL具体的语法形式如下:

操作数 比较操作 ALL(子查询)

这个布尔表达式的意思是子查询结果集中所有的值和给定的操作数做比较操作比较结果为TRUE, 那么整个表达式的结果就为TRUE, 否则整个表达式的结果就为FALSE。比方说下边这个查询:

```
SELECT * FROM e1 WHERE m1 > ALL(SELECT m2 FROM e2);
```

这个查询的意思就是对于e1表的某条记录的m1列的值来说

如果子查询(SELECT m2 FROM e2)的结果集中的所有值都小于m1列的值, 那么整个布尔表达式的值就是TRUE, 否则为FALSE, 也就是说只要m1列的值大于子查询结果集中最大的值, 整个表达式的结果就是TRUE, 所以上边的查询本质上等价于这个查询:

```
SELECT * FROM e1 WHERE m1 > (SELECT MAX(m2) FROM e2);
```

3.4)EXISTS子查询

有的时候我们仅仅需要判断子查询的结果集中是否有记录, 而不在于它的记录具体是个啥, 可以使用把EXISTS或者NOT EXISTS放在子查询语句前边, 就像这样:

```
SELECT * FROM e1 WHERE EXISTS (SELECT 1 FROM e2);
```

对于子查询(SELECT 1 FROM e2)来说, 我们并不关心这个子查询最后到底查询出的结果是什么, 所以查询列表里填*、某个列名, 或者其他啥东西都无所谓, 我们真正关心的是子查询的结果集中是否存在记录。也就是说只要(SELECT 1 FROM e2)这个查询中有记录, 那么整个EXISTS表达式的结果就为TRUE。

1.2.3.3.MySQL对IN子查询的优化

1) 标量子查询、行子查询的执行方式

对于**不相关标量子查询或者行子查询**来说, 它们的执行方式很简单, 比方说下边这个查询语句:

```
SELECT * FROM s1 WHERE order_note = (SELECT order_note FROM s2 WHERE key3 = 'a' LIMIT 1);
```

它的执行方式和我们前面想象的一样:

先单独执行(SELECT order_note FROM s2 WHERE key3 = 'a' LIMIT 1)这个子查询。

然后在将上一步子查询得到的结果当作外层查询的参数,

再执行外层查询SELECT * FROM s1 WHERE order_note= ...。

也就是说, 对于包含不相关的标量子查询或者行子查询的查询语句来说, MySQL会分别独立的执行外层查询和子查询, 就当作两个单表查询就好了。

对于**相关的标量子查询或者行子查询**来说, 比如下边这个查询:

```
SELECT * FROM s1 WHERE order_note = (SELECT order_note FROM s2 WHERE s1.order_no= s2.order_no LIMIT 1);
```

事情也和我们前面想象的一样, 它的执行方式就是这样的:

先从外层查询中获取一条记录, 本例中也就是先从s1表中获取一条记录。

然后从上一步骤中获取的那条记录中找出子查询中涉及到的值，本例中就是从s1表中获取的那条记录中找出s1.order_no列的值，然后执行子查询。

最后根据子查询的查询结果来检测外层查询WHERE子句的条件是否成立，如果成立，就把外层查询的那条记录加入到结果集，否则就丢弃。

再次执行第一步，获取第二条外层查询中的记录，依次类推。

也就是说对于两种使用标量子查询以及行子查询的场景中，MySQL优化器的执行方式并没有什么新鲜的。

2)物化表

对于不相关的IN子查询，比如这样：

```
SELECT * FROM s1 WHERE order_note IN (SELECT order_note FROM s2 WHERE order_no = 'a');
```

我们最开始的感觉就是这种不相关的IN子查询和不相关的标量子查询或者行子查询是一样一样的，都是把外层查询和子查询当作两个独立的单表查询来对待。但是MySQL为了优化IN子查询下了很大力气，所以整个执行过程并不像我们想象的那么简单。

对于不相关的IN子查询来说，如果子查询的结果集中的记录条数很少，那么把子查询和外层查询分别看成两个单独的单表查询效率很高，但是如果单独执行子查询后的结果集太多的话，就会导致这些问题：

- 1、结果集太多，可能内存中都放不下。
- 2、对于外层查询来说，如果子查询的结果集太多，那就意味着IN子句中的参数特别多，这就导致：无法有效的使用索引，只能对外层查询进行全表扫描。

在对外层查询执行全表扫描时，由于IN子句中的参数太多，这会导致检测一条记录是否符合和IN子句中的参数匹配花费的时间太长。

比如说IN子句中的参数只有两个：

```
SELECT * FROM tbl_name WHERE column IN (a, b);
```

这样相当于需要对tbl_name表中的每条记录判断一下它的column列是否符合column = a OR column = b。

在IN子句中的参数比较少时这并不是什么问题，如果IN子句中的参数比较多时，比如这样：

```
SELECT * FROM tbl_name WHERE column IN (a, b, c ..., ...);
```

那么这样每条记录需要判断一下它的column列是否符合column = a OR column = b OR column = c OR ..., 这样性能耗费可就多了。

MySQL的改进是不直接将不相关子查询的结果集当作外层查询的参数，而是将该结果集写入一个临时表里。写入临时表的过程是这样的：

- 1、该临时表的列就是子查询结果集中的列。
- 2、写入临时表的记录会被去重，临时表也是个表，只要为表中记录的所有列建立主键或者唯一索引。

一般情况下子查询结果集不会大的离谱，所以会为它建立基于内存的使用Memory存储引擎的临时表，而且会为该表建立哈希索引。

如果子查询的结果集非常大，超过了系统变量tmp_table_size或者max_heap_table_size，临时表会转而使用基于磁盘的存储引擎来保存结果集中的记录，索引类型也对应转变为B+树索引。

MySQL把这个将子查询结果集中的记录保存到临时表的过程称之为 **物化**（英文名：Materialize）。为了方便起见，我们就把那个存储子查询结果集的临时表称之为物化表。正因为物化表中的记录都建立了索引（基于内存的物化表有哈希索引，基于磁盘的有B+树索引），通过索引执行IN语句判断某个操作数在不在子查询结果集中变得非常快，从而提升了子查询语句的性能。

3)物化表转连接

事情到这就完了？我们还得重新审视一下最开始的那个查询语句：

```
SELECT * FROM s1 WHERE order_note IN (SELECT order_note FROM s2 WHERE order_no = 'a');
```

当我们把子查询进行物化之后，假设子查询物化表的名称为materialized_table，该物化表存储的子查询结果集的列为m_val，那么这个查询

就相当于表s1和子查询物化表materialized_table进行内连接：

```
SELECT s1.* FROM s1 INNER JOIN materialized_table ON order_note = m_val;
```

转化成内连接之后就有意思了，查询优化器可以评估不同连接顺序需要的成本是多少，选取成本最低的那种查询方式执行查询。

我们分析一下上述查询中使用外层查询的表s1和物化表materialized_table进行内连接的成本都是由哪几部分组成的：

1、如果使用s1表作为驱动表的话，总查询成本由下边几个部分组成：

- 物化子查询时需要的成本
- 扫描s1表时的成本

s1表中的记录数量 × 通过m_val = xxx对materialized_table表进行单表访问的成本（我们前边说过物化表中的记录是不重复的，并且为物化表中的列建立了索引，所以这个步骤显然是非常快的）。

2、如果使用materialized_table表作为驱动表的话，总查询成本由下边几个部分组成：

- 物化子查询时需要的成本
- 扫描物化表时的成本

物化表中的记录数量 × 通过order_note= xxx对s1表进行单表访问的成本（如果order_note列上建立了索引，这个步骤还是非常快的）。

MySQL查询优化器会通过运算来选择上述成本更低的方案来执行查询。